Inventory Management System - Technical Documentation

Comprehensive overview for Team Leads and Stakeholders

1. Executive Summary

The Inventory Management System is a full-stack, enterprise-grade web application designed to track products, manage warehouse stock, handle purchase orders from suppliers, and fulfill customer sales. It provides a central hub that maintains an immutable audit trail of all stock movements, preventing stockouts and ensuring data integrity.

2. System Architecture

The project utilizes a strict Modular Monolith architecture. This ensures that features (Accounts, Products, Inventory, Orders) are logically separated for clean code maintenance, while avoiding the overhead and complexity of microservices.

The 3-Tier Architecture consists of:

- Frontend: Server-Side Rendered HTML and CSS templates.
- Backend: Django (Python) handling business logic, validation, and security.
- Database: PostgreSQL for reliable, ACID-compliant data storage.

3. Technology Stack

- Language: Python 3.11+
- Framework: Django 5.1 & Django REST Framework
- Database: PostgreSQL
- Authentication: JWT (JSON Web Tokens) & Session Auth
- Infrastructure: Docker, Nginx, Gunicorn

4. Key Technical Decisions

Why PostgreSQL over SQLite?

Inventory systems require high concurrency. SQLite locks the entire database during writes, which crashes if multiple users attempt to save an order simultaneously. PostgreSQL handles concurrent transactions

seamlessly.

Why Docker & Nginx?

Docker containerizes the application so it runs flawlessly on any server, solving the 'it works on my machine' problem. Nginx acts as a reverse proxy, instantly serving static files (CSS/Images) and shielding the Django backend from direct internet traffic.

Handling Race Conditions

To prevent stock levels from dropping below zero during simultaneous purchases, the system uses Database Transactions (Django's @transaction.atomic and select_for_update). This strictly locks the database row for a fraction of a second, ensuring 100% data integrity.

5. API & Integration

The system exposes a secure RESTful API alongside the HTML dashboard. All API endpoints are protected using JWT (JSON Web Tokens), allowing secure, stateless authentication for future mobile app integrations or external client software.